

Replication and Redundancy

Definition

Replication

Replication means maintaining **multiple copies of data or service instances** across different machines/locations.

Used for:

- Fault-tolerance
- Availability
- Read scaling
- Disaster recovery

Redundancy

Redundancy means having **extra backup components** so if one fails, another continues service.

Examples:

- Extra server
- Extra database replica
- Extra network link
- Backup power supply

Simple Difference

Concept	Meaning
Replication	Copying data/components
Redundancy	Having spare backups

Replication is one form of redundancy.

Why Needed

Without replication/redundancy:

- Single server crash = outage
- Data loss risk
- Slow reads
- No disaster recovery

With them:

- Higher uptime
- Better fault-tolerance
- Faster reads
- Safer operations

Basic Example

3 database nodes:

- 1 Leader (Primary)
- 2 Followers (Replicas)

If one replica fails → system still works

If leader fails → failover possible

Leader-Follower Model

Leader

Source of truth.

Handles:

- Writes
- Updates
- Deletes

Followers / Replicas

Receive copied data from leader.

Handle:

- Reads
- Backup
- Failover candidate

Example Architecture

None

Client

|

Load Balancer

|

Leader DB

/

\

Replica1 Replica2

Replica Placement Strategies

1. Same Data Center

- Lower latency
- Fast replication
- Easier management

But DC outage affects all.

2. Different Rack in Same DC

Protects against:

- Rack failure
- Power zone issue

3. Multi Data Center / Multi Region

Example:

- Mumbai
- Singapore
- Frankfurt

Best fault-tolerance, but slower replication.

Tradeoff: Performance vs Reliability

Placement	Performance	Fault-Tolerance
Same server rack	High	Low
Different rack	Medium	Medium
Different region	Lower	High

Read vs Write Behavior

Writes

Usually go to leader.

Can slow down if synchronous remote replication needed.

Reads

Can go to replicas.

Benefits:

- Scale reads
- Reduce leader load

Example

1000 read QPS, 100 write QPS

Use:

- 1 leader
- 3 read replicas

Now reads distributed.

Common Replication Types

1. Synchronous Replication

Leader waits for replicas to confirm.

Pros:

- Strong consistency
- No data loss

Cons:

- Higher latency

2. Asynchronous Replication

Leader responds immediately, replicas update later.

Pros:

- Fast writes

Cons:

- Replica lag
- Possible data loss on crash

Failover Example

Leader crashes:

1. Detect failure
2. Promote replica to leader
3. Redirect traffic

Tools often used:

Kubernetes

Patroni

Real-World Examples

MySQL

Primary + replicas.

PostgreSQL

Streaming replication.

Apache Cassandra

Peer-to-peer replication.

Hadoop Distributed File System

Default replication factor = 3.

HDFS Example

Each block stored on 3 machines.

If one machine fails:

- Data still available from other copies.

Interview Points

When discussing replication mention:

1. Leader-follower setup
2. Read replicas
3. Sync vs async replication
4. Cross-region replicas
5. Automatic failover

6. Replica lag monitoring

Common Problems

Replica Lag

Follower behind leader.

May return stale data.

Split Brain

Two leaders active simultaneously.

Causes corruption.

Network Partition

Nodes cannot communicate.

Replication vs Backup

Replication	Backup
Live copy	Historical restore point
High availability	Disaster recovery

Fast failover	Restore deleted/corrupt data
---------------	------------------------------

Need both.

One-Line Memory Trick

Replication keeps copies alive; redundancy keeps service alive.